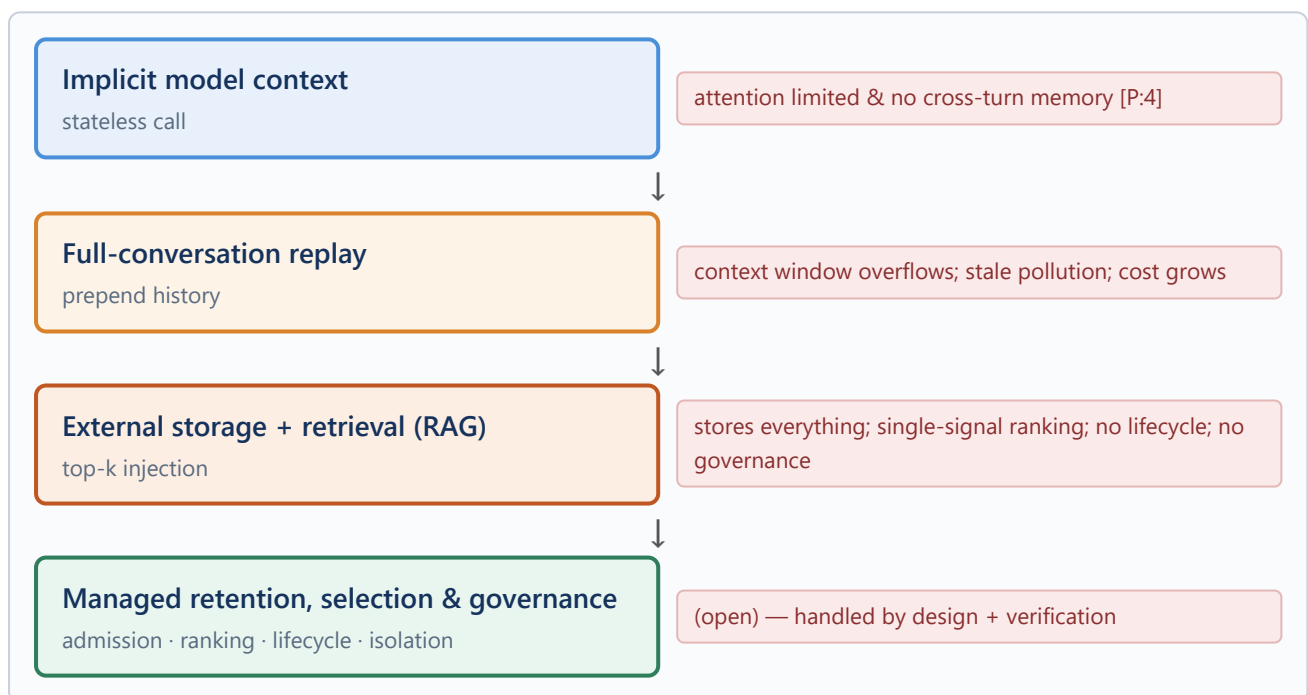


Historical Timeline — How One Bottleneck Creates the Next Approach

Deliverable 1, Part 2. The chain in the form **approach** → **observed bottleneck** → **next approach**, from implicit model context to managed conversational retention. This document explains *why* each step was forced. Claim tags: [P:n] paper-supported (see `sources.md`); [A: ...] assumption to be validated by the naive baseline (Deliverable 3); [o] observed in practice.

0. The shape of the argument

The timeline is not a list of inventions. It is a chain of forced moves: each approach works until a structural limit — a *bottleneck* — makes it unsustainable, and the bottleneck dictates the next approach. The reader who accepts the chain must accept that the final capability set is forced, not chosen.



1. Stage 0 — Implicit model context (the stateless call)

What it is. A model call attends only to the tokens in its context window. Nothing about previous calls is available unless it was explicitly placed in the window. This is structural: attention is computed over a finite context, and there is no external state in the model itself. [P:4]

Why it exists. It is the simplest possible computation and delivers impressive single-turn quality. For short, self-contained queries it is sufficient.

The bottleneck (observed). Every turn is amnesiac:

- No cross-turn memory: a user's facts and preferences vanish between calls.
- No cross-session memory: continuity across days is impossible.
- Nothing can be *chosen* to be remembered — nothing is remembered at all.

What the bottleneck forces. If the model cannot hold state, state must live *outside* the call — on disk. The next stage is forced: write the conversation down and put it back in later.

2. Stage 1 — Full-conversation replay

What it is. Store the whole user history and prepend it to each new turn, so the model can "see" the past. It is the naive first answer to stage 0's bottleneck.

Why it is attractive. Trivial to implement, lossless with respect to what was said, and it demonstrably improves continuity over pure statelessness. The historical record for replay-based approaches is long [O].

The bottleneck (observed, structural):

1. **Context-window overflow.** The window is finite [P:4]. A month of conversation cannot fit; when it is truncated, the oldest (often most important) content silently disappears — the same forgetting it was supposed to fix.
2. **Polluted context.** Replay is indiscriminate: every throwaway remark, contradiction, and stale claim is re-injected with equal weight. The model has no reason to prefer a current statement over an older one.
3. **Unbounded cost and latency.** Token cost and per-token processing grow linearly with history length [P:5], and degrade p95 latency.
4. **No persistence.** Replay works only within the window; it does not and cannot govern what *should* be retained — everything is retained, which is equivalent to nothing is curated.

What the bottleneck forces. The next stage must (a) store only what is worth keeping and (b) *retrieve* a relevant, bounded subset on demand — instead of replaying everything. This is the move to external storage plus retrieval.

3. Stage 2 — External storage and retrieval (retrieval augmentation)

What it is. Content is written to an external store and, at query time, a retriever selects a relevant subset to inject into the context window. Retrieval-Augmented Generation (RAG) makes this the standard pattern for grounding generation in a large corpus [P:5]; iterative retrieval variants like RETRO pushed the same idea to the full document scale [P:6]; and Memorizing Transformers integrated retrieval *into* the model's internal memory [P:7].

Why it works. It caps the context to a bounded, relevant subset. It fixes the worst of Stage 1: no unbounded truncation, no unbounded cost, retrieval is trained or tuned against task utility.

** — what is still wrong:**

1. **It stores everything, indiscriminately.** The store accepts whatever passes through; no *admission* judgment decides whether something is worth keeping. Transient chat, sensitive data, and durable preferences compete for the same index.
2. **Ranking is a single signal.** Typical first implementations rank by vector similarity only. Relevance, recency, importance, and confidence are not balanced; a "plausible but wrong" memory outranks the correct one for preference-laden queries. [A: single-signal ranking failures]
3. **No lifecycle.** Nothing is consolidated, decayed, corrected, or deleted. A superseded preference remains retrievable forever and is returned as if current — replay's pollution problem returns through the retrieval door. [P:8][P:9]
4. **No governance.** Personal data is stored without an admission rule, retained without policy, and can be retrieved across tenants if the index is not tenant-scoped. [A: PII admission + isolation]
5. **Context construction is naive.** Retrieved chunks are concatenated with no budget discipline, no ordering, no fallback when nothing is relevant. [P:9]

What the bottleneck forces. Storage is necessary but not sufficient. The system must decide *what* to store, *which* to retrieve, *how much* to inject, *how long* to keep, and *who* may see it. In short: the retrieval-index view must expand a full memory layer — admission, representation, ranking, context budgeting, lifecycle, isolation, observability.

4. Stage 3 — From retrieval store to managed memory

What it now must be. Retrieval-only systems accumulate data; the literature's next steps are explicitly about *managing the accumulation*:

- **Lifecycle and consolidation.** Generative Agents maintain an observation→reflection→summarization pipeline that consolidates a growing stream into durable higher-level memory, instead of replaying every observation raw [P:8].
- **Memory hierarchy and paging.** MemGPT treats context as the "main memory" of an OS and pages relevant memory in and out against a budget, with storage and retrieval controlled by the agent itself — not by naive concatenation [P:9].
- **To-element architectures.** External read/write memory addressed by content — Memory Networks [P:3] and End-to-End Memory Networks [P:2] — established that a trained controller can attend over an external store; Neural Turing Machines [P:1] tied an external read/write tape to a differentiable controller.
- **Retrieval and ranking.** Indexing and similarity search (FAISS [P:10], HNSW [P:11]) provide the retrieval substrate, but the *ranking policy* (which signals, how weighted, how conflicts resolved) remains a system design decision.

The remaining pressures. These historical moves converge on a single realization: **conversational memory is an engineered system composed of admission, representation, retrieval+ranking, context construction, lifecycle, governance, and observation** — with each stage of the timeline visible as one of its components. The naive baseline (Deliverable 3) will measure the failures of Stage 2 in our workload so that Stage 3's design can be justified per-component and compared against measurement, not assertion.

5. Timeline summary

Stage	Approach	Works until (bottleneck)	Forces next approach
0	Implicit model context	Amnesia across calls/sessions	External state on disk
1	Full-conversation replay	Context overflow + pollution + cost	Bounded, selective retrieval
2	External store + retrieval (RAG)	Store-everything; weak ranking; no lifecycle; no governance	Admission + selection + lifecycle + isolation
3	Managed memory (hierarchy, reflection, paging, governance)	(open) — handled by design + verification	Deliverables 4-6

6. What this means for the problem reconstruction

The timeline's conclusion is the same conclusion reached independently from first principles in `first_principles.md`: a credible answer cannot be "just retrieval" or "just replay" — it must include decisions [admission], limits [budget], time [lifecycle], and boundaries [isolation]. The design document (Deliverable 4) must explain how Stage 3's system carries out each of these, and the experiment (Deliverable 3) must demonstrate why each of Stage 2's failures is real for this project's workload.

End of historical timeline.